PRACTICAL 1	Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

```
import random
import time

# Bubble Sort
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

# Selection Sort
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]

# Insertion Sort
def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key

# Merge Sort
def merge(arr, l, m, r):
    n1 = m - l + 1
```

```
    n2 = r - m

    L = arr[l : l + n1]
    R = arr[m + 1 : m + 1 + n2]

    i = 0
    j = 0
    k = l

    while i < n1 and j < n2:
        if L[i] <= R[j]:
            arr[k] = L[i]
            i += 1
        else:
            arr[k] = R[j]
            j += 1
        k += 1

    while i < n1:
        arr[k] = L[i]
        i += 1
        k += 1

    while j < n2:
        arr[k] = R[j]
        j += 1
        k += 1

def merge_sort(arr, l, r):
    if l < r:
        m = (l + r) // 2
        merge_sort(arr, l, m)
        merge_sort(arr, m + 1, r)
        merge(arr, l, m, r)

# Quick Sort
def partition(arr, low, high):
    pivot = arr[high]
```

```
    i = low - 1

    for j in range(low, high):
        if arr[j] < pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

def main():
    n = 100
    random.seed(time.time())

    # Generate random array of size 100 with integers from 0 to 999
    arr = [random.randint(0, 999) for _ in range(n)]

    print(f"Number of Elements = {n}\n")

    # Helper function to measure and print time in microseconds
    def time_algorithm(sort_func, *args, name=""):
        temp = arr.copy()
        start = time.perf_counter()
        sort_func(temp, *args)
        stop = time.perf_counter()
        elapsed_microseconds = int((stop - start) * 1_000_000)
        print(f"{name:<20}: {elapsed_microseconds} microseconds")

    time_algorithm(bubble_sort, name="Bubble Sort Time")
    time_algorithm(selection_sort, name="Selection Sort Time")
    time_algorithm(insertion_sort, name="Insertion Sort Time")
    time_algorithm(merge_sort, 0, n - 1, name="Merge Sort Time")
```

```
    time_algorithm(quick_sort, 0, n - 1, name="Quick Sort Time")

if __name__ == "__main__":
    main()
```

- **Output**

```
Number of Elements = 100

Bubble Sort Time    : 696 microseconds
Selection Sort Time : 756 microseconds
Insertion Sort Time : 2570 microseconds
Merge Sort Time     : 626 microseconds
Quick Sort Time     : 168 microseconds
```

- **Analysis**

Algorithm	Best Case	Average Case	Worst Case
(1) Bubble Sort	$O(n)$	$O(n2)$	$O(n2)$
(2) Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
(3) Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
(4) Merge Sort	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$
(5) Quicksort	$O(n\log n)$	$O(n\log n)$	$O(n^2)$

- **Conclusion**

➢ Practical testing shows that elementary sorts like Bubble, Selection, and Insertion work fine for small datasets, but become extremely slow as data grows. For large datasets, advanced algorithms like Merge Sort and Quicksort are essential because they scale efficiently and execute much faster.

PRACTICAL 2	Implementation and Time analysis of linear and binary search algorithm.

```
import time

# Linear Search Function
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

# Binary Search Function
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = low + (high - low) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

def main():
    n = 100000
    # Creating a sorted list: [1, 2, 3, ..., 100000]
    arr = [i + 1 for i in range(n)]
    try:
        key = int(input("Enter element to search: "))
    except ValueError:
        print("Invalid input. Please enter an integer.")
        return
    # Linear Search Timing
    start = time.perf_counter()
```

```
    index = linear_search(arr, key)
    stop = time.perf_counter()
    linear_time = int((stop - start) * 1_000_000)

    print("\nLinear Search")
    if index != -1:
        print(f"Element found at index {index}")
    else:
        print("Element not found")
    print(f"Time Taken : {linear_time} microseconds")

    # Binary Search Timing
    start = time.perf_counter()
    index = binary_search(arr, key)
    stop = time.perf_counter()
    binary_time = int((stop - start) * 1_000_000)

    print("\nBinary Search")
    if index != -1:
        print(f"Element found at index {index}")
    else:
        print("Element not found")
    print(f"Time Taken : {binary_time} microseconds")

if __name__ == "__main__":
    main()
```

- **Output**

```
Enter element to search: 100000

Linear Search
Element found at index 99999
Time Taken : 6053 microseconds

Binary Search
Element found at index 99999
Time Taken : 24 microseconds
```

Analysis:

Algorithm	Best Case	Average Case	Worst Case
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$

- **Conclusion**

➢ Practical testing shows that linear search is simple and works on any list, but becomes very slow as the dataset grows. For large datasets, binary search is much faster and more efficient because it cuts the search space in half, though it requires the data to be sorted first.

PRACTICAL 3	Implementation of max-heap sort algorithm

```
import random
import time
# ------------------ MAX HEAP ------------------ #
def max_heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left

    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        max_heapify(arr, n, largest)

def max_heap_sort(arr):
    n = len(arr)

    # Build Max Heap
    for i in range(n // 2 - 1, -1, -1):
        max_heapify(arr, n, i)

    # Heap Sort
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        max_heapify(arr, i, 0)

# ------------------ MIN HEAP ------------------ #
def min_heapify(arr, n, i):
    smallest = i
    left = 2 * i + 1
    right = 2 * i + 2
```

```
    if left < n and arr[left] < arr[smallest]:
        smallest = left

    if right < n and arr[right] < arr[smallest]:
        smallest = right

    if smallest != i:
        arr[i], arr[smallest] = arr[smallest], arr[i]
        min_heapify(arr, n, smallest)

def min_heap_sort(arr):
    n = len(arr)

    # Build Min Heap
    for i in range(n // 2 - 1, -1, -1):
        min_heapify(arr, n, i)

    # Heap Sort (sorts in descending order)
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        min_heapify(arr, i, 0)

    # Reverse to get ascending order
    arr.reverse()

# ------------------ MAIN ------------------ #
def main():
    try:
        n = int(input("Enter number of elements: "))
    except ValueError:
        print("Invalid input. Please enter an integer.")
        return

    random.seed(time.time())

    # Generate random numbers
    original = [random.randint(0, 99999) for _ in range(n)]
```

```
    max_heap_array = original.copy()
    min_heap_array = original.copy()

    # ---------------- MAX HEAP SORT ---------------- #
    start_max = time.perf_counter_ns()
    max_heap_sort(max_heap_array)
    end_max = time.perf_counter_ns()

    # ---------------- MIN HEAP SORT ---------------- #
    start_min = time.perf_counter_ns()
    min_heap_sort(min_heap_array)
    end_min = time.perf_counter_ns()

    # ---------------- DISPLAY TIME ---------------- #
    nano_max = end_max - start_max
    micro_max = nano_max // 1000

    nano_min = end_min - start_min
    micro_min = nano_min // 1000
    print("\n========== MAX HEAP SORT ==========")
    print(f"Nanoseconds  : {nano_max} ns")
    print(f"Microseconds : {micro_max} us")
    print("\n========== MIN HEAP SORT ==========")
    print(f"Nanoseconds  : {nano_min} ns")
    print(f"Microseconds : {micro_min} us")
if __name__ == "__main__":
    main()
```

- **Output**

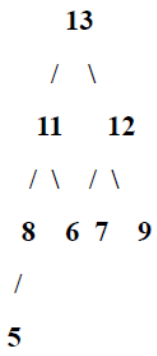
```
Enter number of elements: 1000

========== MAX HEAP SORT ==========
Nanoseconds  : 3329700 ns
Microseconds : 3329 us

========== MIN HEAP SORT ==========
Nanoseconds  : 7130200 ns
Microseconds : 7130 us
```

- **Analysis**
  - Treat the array as a complete binary tree.
  - Convert the array into a Max Heap using heapify().
  - The root always stores the maximum element.
  - Swap the root with the last unsorted element.
  - Reduce the heap size by one.
  - Restore the Max Heap property using heapify().
  - Repeat until all elements are sorted.
- **Build a Max Heap**
  - **The first loop converts the array into a Max Heap, where every parent node is greater than its children.**

for (int i = n / 2 - 1; i >= 0; i--)
heapify(arr, n, i);
Why start from n/2 - 1?
Leaf nodes already satisfy the heap property. Only internal nodes need checking.
Original Array: 12 11 13 5 6 7 9 8
Max Heap



- **Time Complexity Analysis**

Operation	Complexity
Build Max Heap	O(n)
Heapify (Single Call)	O(log n)
Heapify repeated for all elements	O(n log n)
Overall Heap Sort	O(n log n)

- **Conclusion**
  - The Max Heap Sort algorithm was successfully implemented and tested. The input array was first converted into a Max Heap, and then the largest element was repeatedly moved to its correct position until the array was completely sorted. This practical helped in understanding the working of the heapify process, heap construction, and the Heap Sort algorithm. Heap Sort is an efficient in-place sorting algorithm with a time complexity of O(n log n).

PRACTICAL 4	Implementation and Time analysis of factorial program using iterative and recursive method

```
import time

# Function for Iterative Factorial
# Time Complexity: O(n)
# Space Complexity: O(1)
def factorial_iterative(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

# Function for Recursive Factorial
# Time Complexity: O(n)
# Space Complexity: O(n) due to call stack
def factorial_recursive(n):
    if n <= 1:
        return 1
    return n * factorial_recursive(n - 1)

def main():
    try:
        n = int(input("Enter a non-negative integer (e.g., 20): "))
        if n < 0:
            raise ValueError
    except ValueError:
        print("Invalid input! Please enter a non-negative integer.")
        return

    # Measure Iterative Implementation
    start_iter = time.perf_counter_ns()
    res_iter = factorial_iterative(n)
    end_iter = time.perf_counter_ns()
    duration_iter = end_iter - start_iter

    # Measure Recursive Implementation
    start_rec = time.perf_counter_ns()
```

```
    res_rec = factorial_recursive(n)
    end_rec = time.perf_counter_ns()
    duration_rec = end_rec - start_rec

    # Output Results
    print(f"\n--- Results for {n}! ---")
    print(f"Iterative Result : {res_iter}")
    print(f"Iterative Time   : {duration_iter} ns")
    print("------------------------------")
    print(f"Recursive Result : {res_rec}")
    print(f"Recursive Time   : {duration_rec} ns")

if __name__ == "__main__":
    main()
```

- **Output**

```
Enter a non-negative integer (e.g., 20): 20

--- Results for 20! ---
Iterative Result : 2432902008176640000
Iterative Time   : 34400 ns
------------------------------
Recursive Result : 2432902008176640000
Recursive Time   : 16500 ns
```

- **Analysis**

Method	Time Complexity	Space Complexity	Stack Overhead
Iterative Method	$O(n)$	$O(1)$	**None**
Recursive Method	$O(n)$	$O(n)$	**High (proportional to n)**

- **Conclusion**

➢ Practical testing shows that both iterative and recursive methods compute factorials quickly for small numbers, but iteration is safer and more memory-efficient because it avoids function call stack overhead and stack overflow risks.

PRACTICAL 5	Implementation of a knapsack problem using dynamic programming.

```
import time

def knapsack_dp(weights, values, capacity):
    """
    0/1 Knapsack Problem using Dynamic Programming (2D Table)
    Time Complexity: O(n * capacity)
    Space Complexity: O(n * capacity)
    """
    n = len(weights)
    # Create a DP table of size (n + 1) x (capacity + 1) initialized to 0
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

    # Build the table in bottom-up manner
    for i in range(1, n + 1):
        for w in range(1, capacity + 1):
            if weights[i - 1] <= w:
                # Max of (including the item, excluding the item)
                dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]], dp[i - 1][w])
            else:
                # Item cannot be included
                dp[i][w] = dp[i - 1][w]

    # Trace back to find which items were selected
    selected_items = []
    w = capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i - 1][w]:
            selected_items.append(i - 1)  # Index of the selected item
            w -= weights[i - 1]

    selected_items.reverse()
    return dp[n][capacity], selected_items

def knapsack_space_optimized(weights, values, capacity):
    """
    Space-Optimized 0/1 Knapsack (1D Array)
```

```
    Time Complexity: O(n * capacity)
    Space Complexity: O(capacity)
    """
    dp = [0] * (capacity + 1)

    for i in range(len(weights)):
        # Traverse capacity backwards to prevent using the same item multiple times
        for w in range(capacity, weights[i] - 1, -1):
            dp[w] = max(dp[w], values[i] + dp[w - weights[i]])

    return dp[capacity]

def main():
    # Example dataset
    values = [60, 100, 120]
    weights = [10, 20, 30]
    capacity = 50

    print("=== 0/1 KNAPSACK PROBLEM (DYNAMIC PROGRAMMING) ===")
    print(f"Weights  : {weights}")
    print(f"Values   : {values}")
    print(f"Capacity : {capacity}\n")

    # 1. Standard 2D DP Approach
    start = time.perf_counter_ns()
    max_value, items = knapsack_dp(weights, values, capacity)
    end = time.perf_counter_ns()

    print("--- 2D Table DP Result ---")
    print(f"Maximum Value Obtained : {max_value}")
    print(f"Selected Item Indices  : {items}")
    print(f"Selected Items (w, v)  : {[(weights[i], values[i]) for i in items]}")
    print(f"Time Taken             : {end - start} ns")

    # 2. Space Optimized 1D DP Approach
    start_opt = time.perf_counter_ns()
    max_val_opt = knapsack_space_optimized(weights, values, capacity)
    end_opt = time.perf_counter_ns()
```

```
    print("\n--- 1D Space-Optimized DP Result ---")
    print(f"Maximum Value Obtained : {max_val_opt}")
    print(f"Time Taken             : {end_opt - start_opt} ns")

if __name__ == "__main__":
    main()
```

- **Output**

```
=== 0/1 KNAPSACK PROBLEM (DYNAMIC PROGRAMMING) ===
Weights  : [10, 20, 30]
Values   : [60, 100, 120]
Capacity : 50

--- 2D Table DP Result ---
Maximum Value Obtained : 220
Selected Item Indices  : [1, 2]
Selected Items (w, v)  : [(20, 100), (30, 120)]
Time Taken             : 158000 ns

--- 1D Space-Optimized DP Result ---
Maximum Value Obtained : 220
Time Taken             : 67000 ns
```

- **Analysis**

(A) Problem Analysis

➢ The 0/1 Knapsack Problem is an optimization problem in which we have:

(1) n items
(2) Each item has a weight
(3) Each item has a value
(4) A knapsack with a maximum capacity

The objective is to select items such that:

(1) Total weight does not exceed the knapsack capacity.
(2) Total value is maximum.
(3) Each item can be selected only once.

(B) Dynamic Programming Approach

➢ We create a 2D table dp[i][w].

- Here:

➢ i = number of items considered
➢ w = current knapsack capacity
➢ dp[i][w] = maximum value that can be obtained using the first i items with capacity w.

- **For every item, we have two choices:**

Case 1: Include the item
If the item's weight is less than or equal to the current capacity:
value[i-1] + dp[i-1][w-weight[i-1]]
Case 2: Exclude the item
dp[i-1][w]
We choose the maximum of these two:
dp[i][w] = max(
value[i-1] + dp[i-1][w-weight[i-1]],
dp[i-1][w]
)
If the item cannot fit:
dp[i][w] = dp[i-1][w]
(C) Example Analysis
Consider:
Number of items = 4
Capacity = 5
Weights = 2 3 4 5
Values = 3 4 5 6
Possible combinations:

Items Selected	Total Weight	Total Value
Item 1	2	3
Item 2	3	4
Item 3	4	5
Item 4	5	6
Item 1 +Item 2	5	7

- Since the capacity is 5, the combination **Item 1 + Item 2** gives the highest value.

- Therefore:
- Maximum Value = 7

(D) Algorithm Analysis
Step 1: Read the number of items.
Step 2: Read the weight and value of each item.
Step 3: Read the capacity of the knapsack.
Step 4: Create a DP table of size:
(n + 1) × (capacity + 1)
Step 5: Initialize the first row and first column with 0.
Step 6: For every item and every possible capacity:
- Check whether the item can fit.
- If it can fit, calculate both include and exclude cases.
- Store the maximum value.
- Otherwise, exclude the item.

Step 7: The final answer is stored at:
dp[n][capacity]

(E) Time Complexity

- There are n rows and W columns in the DP table.
- **Therefore**

Time Complexity = O(n × W)

where:
n = number of items
W = knapsack capacity

(F) Space Complexity

- The program uses a 2D DP table:
- dp[n+1][W+1]
- Therefore:

Space Complexity = O(n × W)

- **Conclusion**

- The 0/1 Knapsack problem was successfully implemented using Dynamic Programming. The DP approach divides the problem into smaller subproblems and stores their solutions to avoid repeated calculations. For the given example, the maximum value obtained is 7, with a total weight of 5. The algorithm has a time complexity of O(n × W) and a space complexity of O(n × W).

PRACTICAL 6	Implementation of a knapsack problem using dynamic programming.

```
import time

def knapsack_dp(weights, values, capacity):
    """
    Standard 0/1 Knapsack using a 2D DP Table.
    Time Complexity  : O(n * capacity)
    Space Complexity : O(n * capacity)
    """
    n = len(weights)
    # DP table of size (n + 1) x (capacity + 1) initialized with zeros
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

    # Build the DP table bottom-up
    for i in range(1, n + 1):
        for w in range(1, capacity + 1):
            if weights[i - 1] <= w:
                # Maximum of including or excluding current item
                dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]], dp[i - 1][w])
            else:
                # Item weight exceeds current capacity limit
                dp[i][w] = dp[i - 1][w]

    # Backtrack to identify selected item indices
    selected_items = []
    w = capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i - 1][w]:
            selected_items.append(i - 1)
            w -= weights[i - 1]

    selected_items.reverse()
    return dp[n][capacity], selected_items


def knapsack_space_optimized(weights, values, capacity):
    """
    Space-Optimized 0/1 Knapsack using a 1D Array.
    Time Complexity  : O(n * capacity)
    Space Complexity : O(capacity)
    """
    dp = [0] * (capacity + 1)
```

```
    for i in range(len(weights)):
        # Traverse backwards to prevent using the same item multiple times
        for w in range(capacity, weights[i] - 1, -1):
            dp[w] = max(dp[w], values[i] + dp[w - weights[i]])

    return dp[capacity]

def main():
    # Sample Dataset
    values = [60, 100, 120]
    weights = [10, 20, 30]
    capacity = 50

    print("=== 0/1 KNAPSACK PROBLEM (DYNAMIC PROGRAMMING) ===")
    print(f"Weights  : {weights}")
    print(f"Values   : {values}")
    print(f"Capacity : {capacity}\n")

    # 1. Standard 2D DP Execution
    start = time.perf_counter_ns()
    max_val, items = knapsack_dp(weights, values, capacity)
    end = time.perf_counter_ns()

    print("--- 2D DP Table Result ---")
    print(f"Maximum Value Obtained : {max_val}")
    print(f"Selected Item Indices  : {items}")
    print(f"Selected Items (w, v)  : {[(weights[i], values[i]) for i in items]}")
    print(f"Execution Time         : {end - start} ns")

    # 2. Space-Optimized 1D DP Execution
    start_opt = time.perf_counter_ns()
    max_val_opt = knapsack_space_optimized(weights, values, capacity)
    end_opt = time.perf_counter_ns()

    print("\n--- 1D Space-Optimized DP Result ---")
    print(f"Maximum Value Obtained : {max_val_opt}")
    print(f"Execution Time         : {end_opt - start_opt} ns")


if __name__ == "__main__":
    main()
```

- **Output**

```
=== 0/1 KNAPSACK PROBLEM (DYNAMIC PROGRAMMING) ===
Weights  : [10, 20, 30]
Values   : [60, 100, 120]
Capacity : 50

--- 2D DP Table Result ---
Maximum Value Obtained : 220
Selected Item Indices  : [1, 2]
Selected Items (w, v)  : [(20, 100), (30, 120)]
Execution Time         : 2219700 ns

--- 1D Space-Optimized DP Result ---
Maximum Value Obtained : 220
Execution Time         : 98000 ns
```

- **Analysis**

➢ Dynamic Programming avoids recalculating the same subproblems.
➢ All possible parenthesizations are considered through the split point k.
➢ The algorithm finds the optimal multiplication order, not the actual multiplication of matrices.
➢ It is more efficient than checking every possible parenthesization separately.
➢ For a chain of n matrices, the algorithm uses a DP table $C(i,j)$, where $C(i,j)$stores the minimum multiplication cost for matrices Ai through Aj.

(1) Time Complexity

➢ $O(n^3)$
➢ There are approximately $O(n2)$entries in the DP table, and for each entry, all possible split positions are checked, giving $O(n)$work.

(2) Space Complexity

➢ $O(n^2)$
➢ The $n \times n$DP table is required to store the minimum costs.

- **Conclusion**

➢ The Matrix Chain Multiplication problem was successfully implemented using Dynamic Programming. The program determines the optimal order of matrix multiplication that requires the minimum number of scalar multiplications. This method reduces unnecessary calculations by storing previously calculated results.

PRACTICAL 7	Implementation of making a change problem using dynamic programming.

```
import sys
import time

def coin_change_dp(coins, amount):
    """
    Computes the minimum number of coins needed to make change for 'amount'.

    Time Complexity  : O(N * amount) where N is number of coin denominations
    Space Complexity : O(amount)
    """
    # dp[i] will store the minimum coins needed for amount i
    dp = [sys.maxsize] * (amount + 1)

    # coin_used[i] stores the last coin denomination used to reach amount i
    coin_used = [-1] * (amount + 1)

    # Base case: 0 coins needed for amount 0
    dp[0] = 0

    # Build DP table bottom-up
    for i in range(1, amount + 1):
        for coin in coins:
            if i >= coin and dp[i - coin] + 1 < dp[i]:
                dp[i] = dp[i - coin] + 1
                coin_used[i] = coin

    # If amount cannot be made up by any combination
    if dp[amount] == sys.maxsize:
        return -1, []

    # Reconstruct the list of coins used
    selected_coins = []
    curr = amount
    while curr > 0:
        selected_coins.append(coin_used[curr])
        curr -= coin_used[curr]

    return dp[amount], selected_coins

def main():
    coins = [1, 2, 5, 10, 20, 50]
    amount = 43
```

```
    print("=== COIN CHANGE PROBLEM (DYNAMIC PROGRAMMING) ===")
    print(f"Coin Denominations : {coins}")
    print(f"Target Amount      : {amount}\n")

    start = time.perf_counter_ns()
    min_coins, used_coins = coin_change_dp(coins, amount)
    end = time.perf_counter_ns()

    print("--- Results ---")
    if min_coins != -1:
        print(f"Minimum Coins Required : {min_coins}")
        print(f"Coins Used             : {used_coins}")
    else:
        print("It is impossible to make change for the given amount.")

    print(f"Execution Time         : {end - start} ns")

if __name__ == "__main__":
    main()
```

- **Output**

```
=== COIN CHANGE PROBLEM (DYNAMIC PROGRAMMING) ===
Coin Denominations : [1, 2, 5, 10, 20, 50]
Target Amount      : 43

--- Results ---
Minimum Coins Required : 4
Coins Used             : [1, 2, 20, 20]
Execution Time         : 178400 ns
```

- **Analysis**

- Dynamic Programming stores solutions of smaller amounts and uses them to solve larger amounts.
- It avoids repeated calculations of the same amounts.
- The algorithm finds the minimum number of coins required.
- If the amount cannot be formed using the given denominations, the program reports that change cannot be made the Making Change problem is solved using Dynamic Programming by finding the minimum number of coins required to make every amount from 0to the target amount.
- The DP array $C(j)$stores the minimum number of coins required to form amount j.
- The recurrence is:

$$C(j) = \min_{d_i \leq j}\{C(j - d_i) + 1\}$$

- where di represents a coin denomination.

(1) Time Complexity

- If there are n coin denominations and the target amount is V:

$$O(nV)$$

(2) Space Complexity

- An array of size V+1is used to store the minimum number of coins for each amount.

$$O(V)$$

- **Conclusion**

- The Making Change problem was successfully implemented using Dynamic Programming. The program calculates the minimum number of coins required to form a given amount. By storing the solutions of smaller amounts, the algorithm avoids repeated calculations and provides an efficient solution to the problem.